

Reti Logiche

3° Esercitazione: circuiti sequenziali

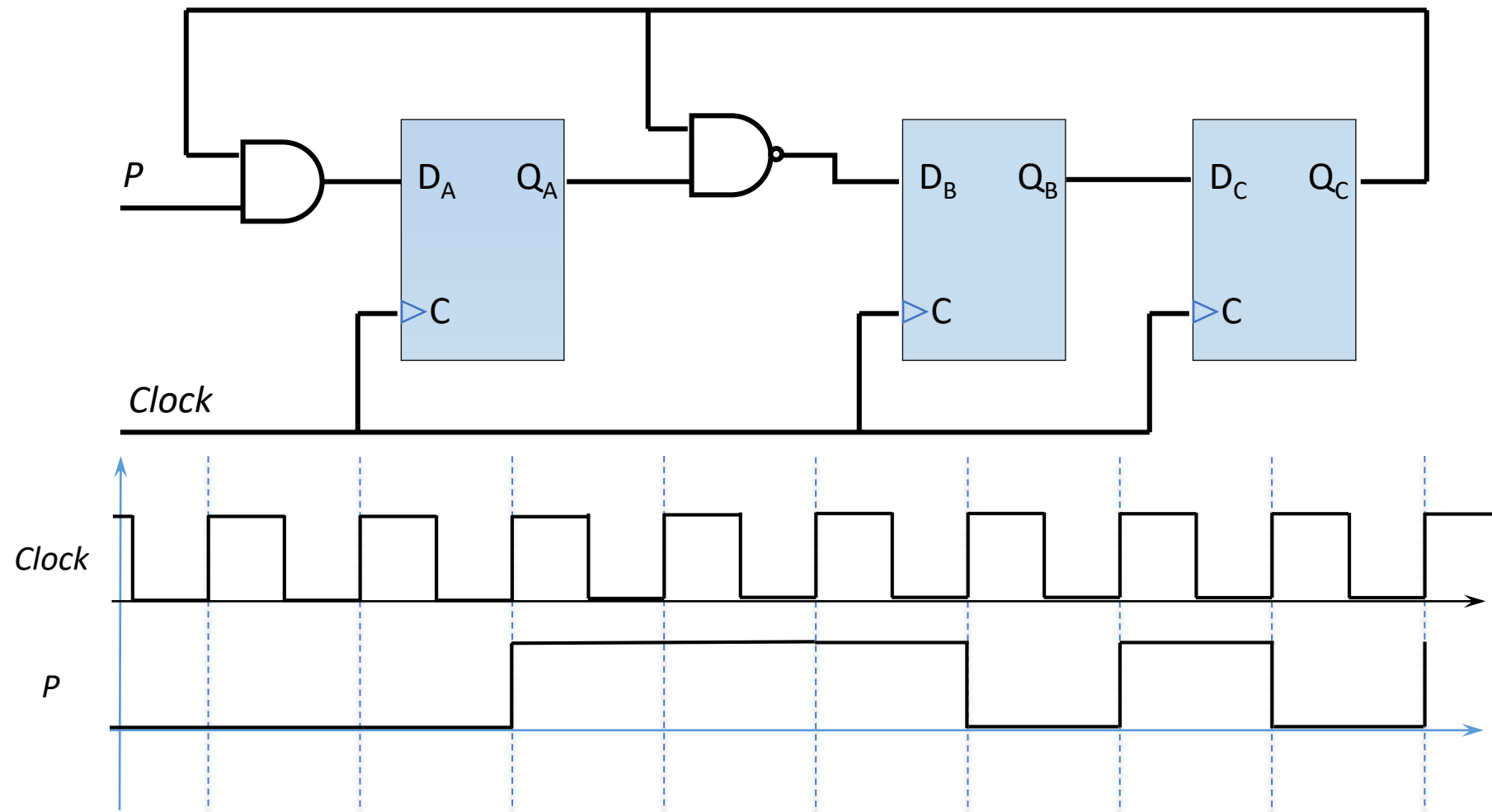
Enrico Manuzzato

Esercizi

- Analisi di un circuito sequenziale
- Contatore di uni
- Misuratore di impulsi

Analisi di un circuito sequenziale

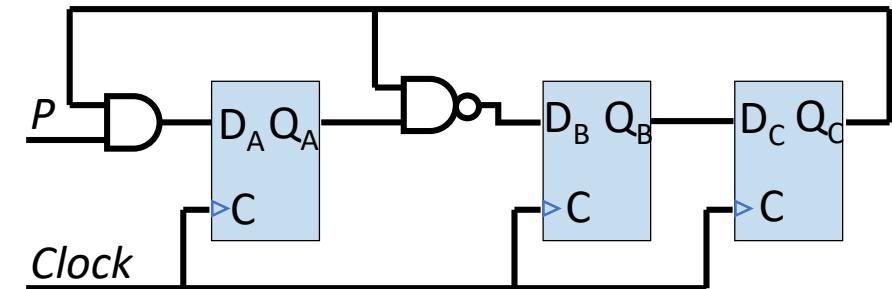
- Dato il seguente circuito sequenziale descriverne il funzionamento e completare il diagramma temporale per i segnali Q_A , Q_B , Q_C al variare di P .



Analisi di un circuito sequenziale

- Posso derivare subito le espressioni per lo stato futuro dato stato presente ed ingresso:
 - $D_A = P \cdot Q_C$
 - $D_B = \overline{Q_A} \cdot Q_C$
 - $D_C = Q_B$
- Compilo la tabella degli stati per $P = 1$ e $P = 0$

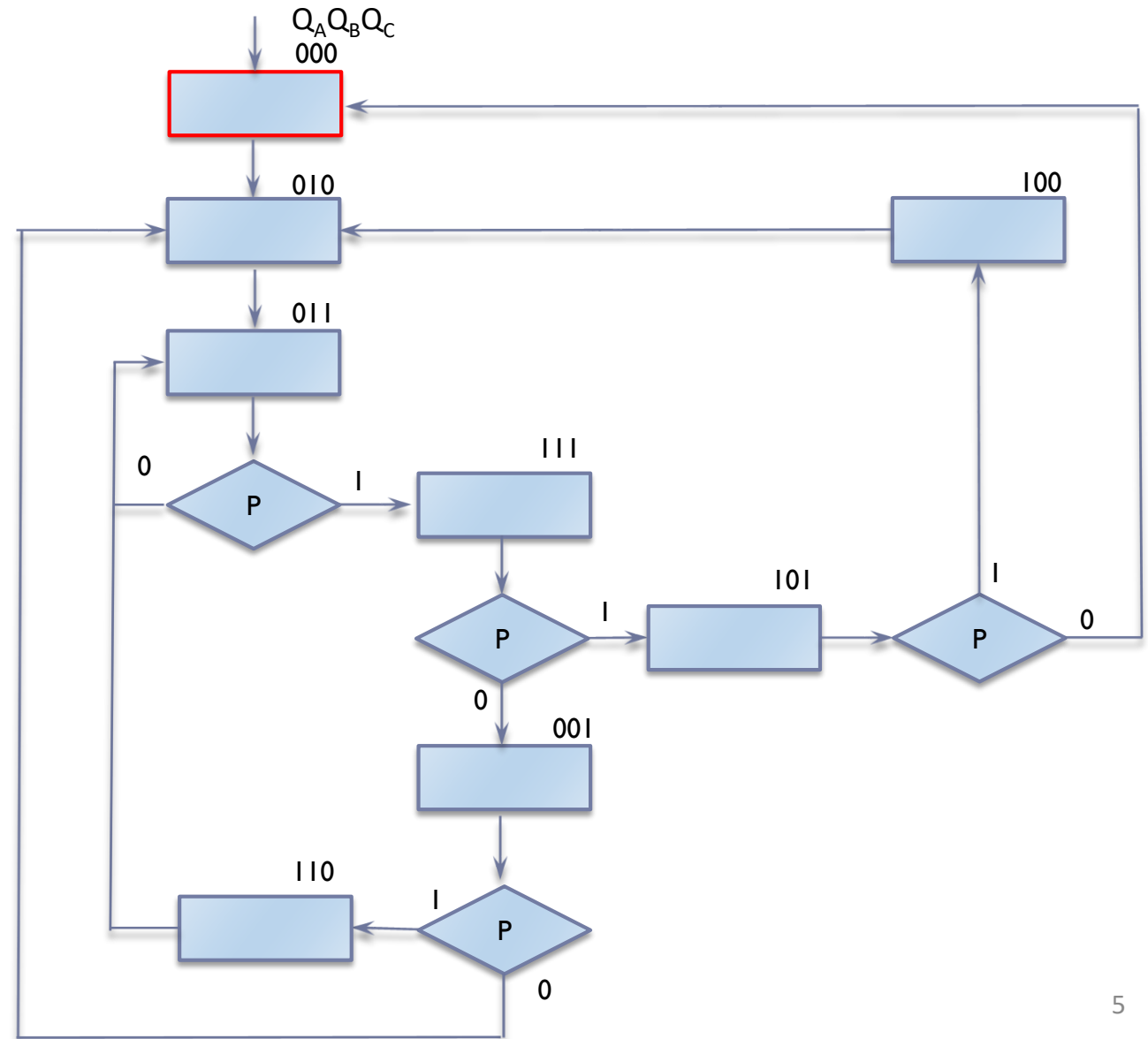
Stato presente			Stato Futuro ($P = 0$)			Stato Futuro ($P = 1$)		
Q_A	Q_B	Q_C	D_A	D_B	D_C	D_A	D_B	D_C
0	0	0	0	1	0	0	1	0
0	0	1	0	1	0	1	1	0
0	1	0	0	1	1	0	1	1
0	1	1	0	1	1	1	1	1
1	0	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0	0
1	1	0	0	1	1	0	1	1
1	1	1	0	0	1	1	0	1



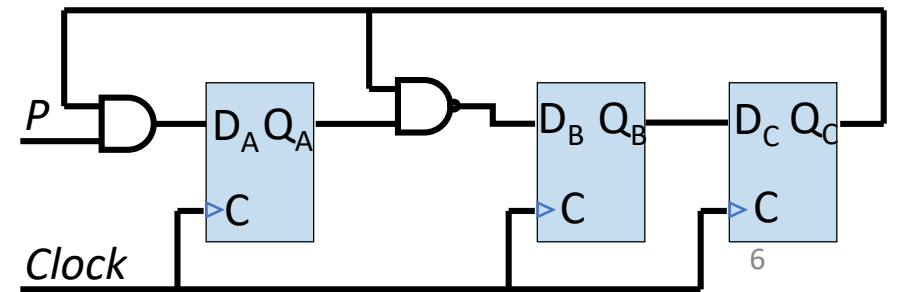
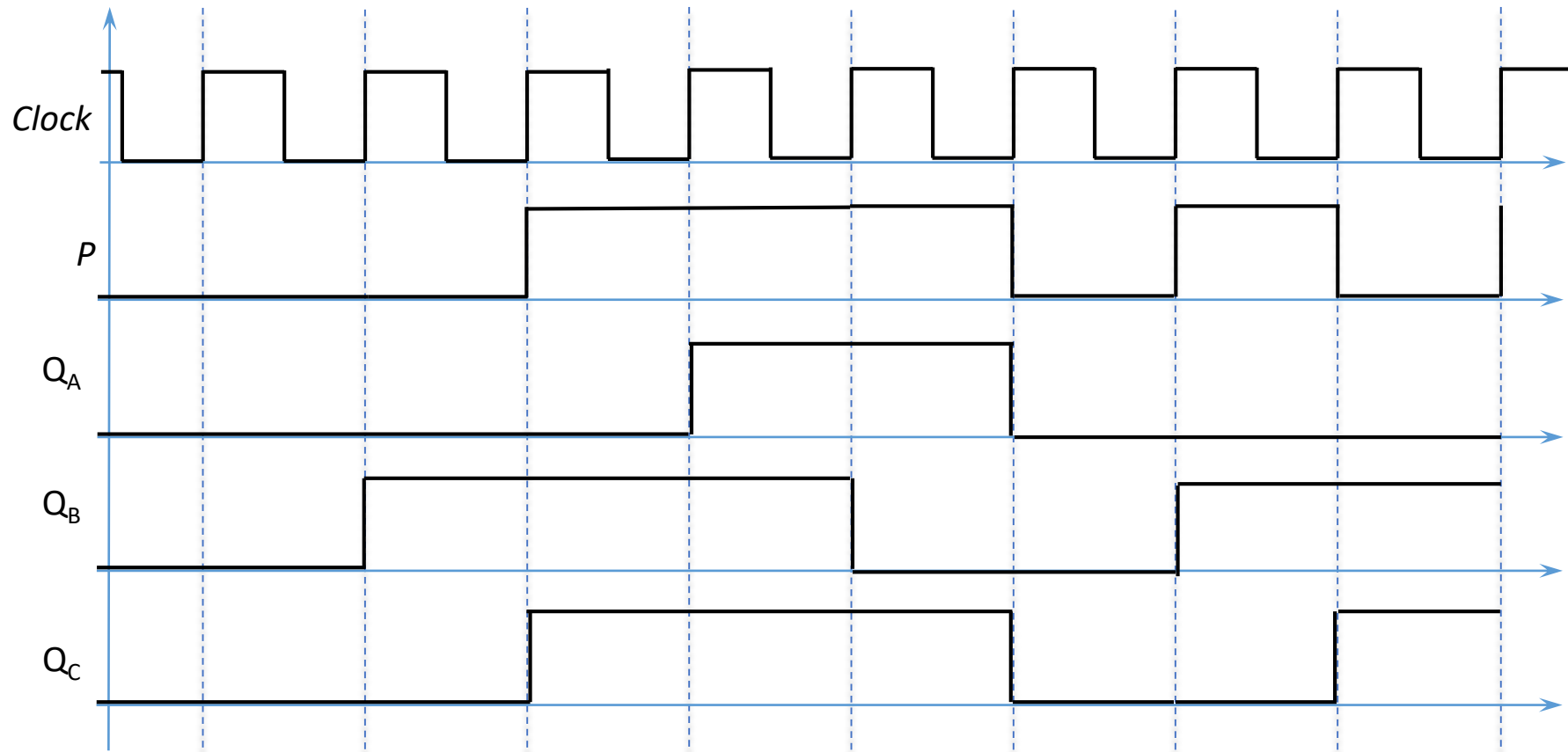
Analisi di un circuito sequenziale

- Diagramma ASM

Stato presente			Stato Futuro ($P = 0$)			Stato Futuro ($P = 1$)		
Q_A	Q_B	Q_C	D_A	D_B	D_C	D_A	D_B	D_C
0	0	0	0	1	0	0	1	0
0	0	1	0	1	0	1	1	0
0	1	0	0	1	1	0	1	1
0	1	1	0	1	1	1	1	1
1	0	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0	0
1	1	0	0	1	1	0	1	1
1	1	1	0	0	1	1	0	1

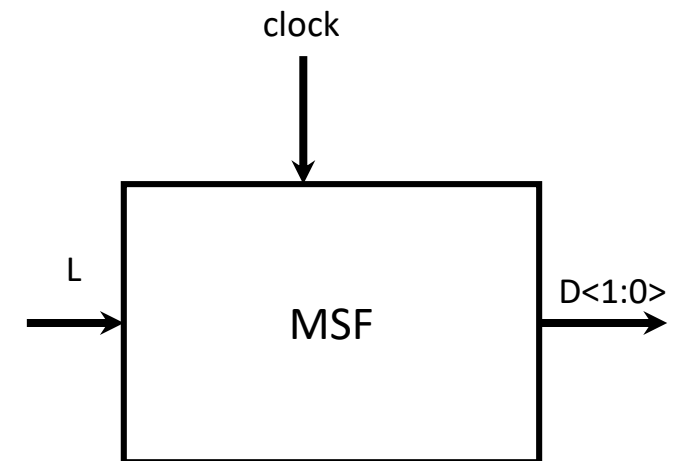


Analisi di un circuito sequenziale



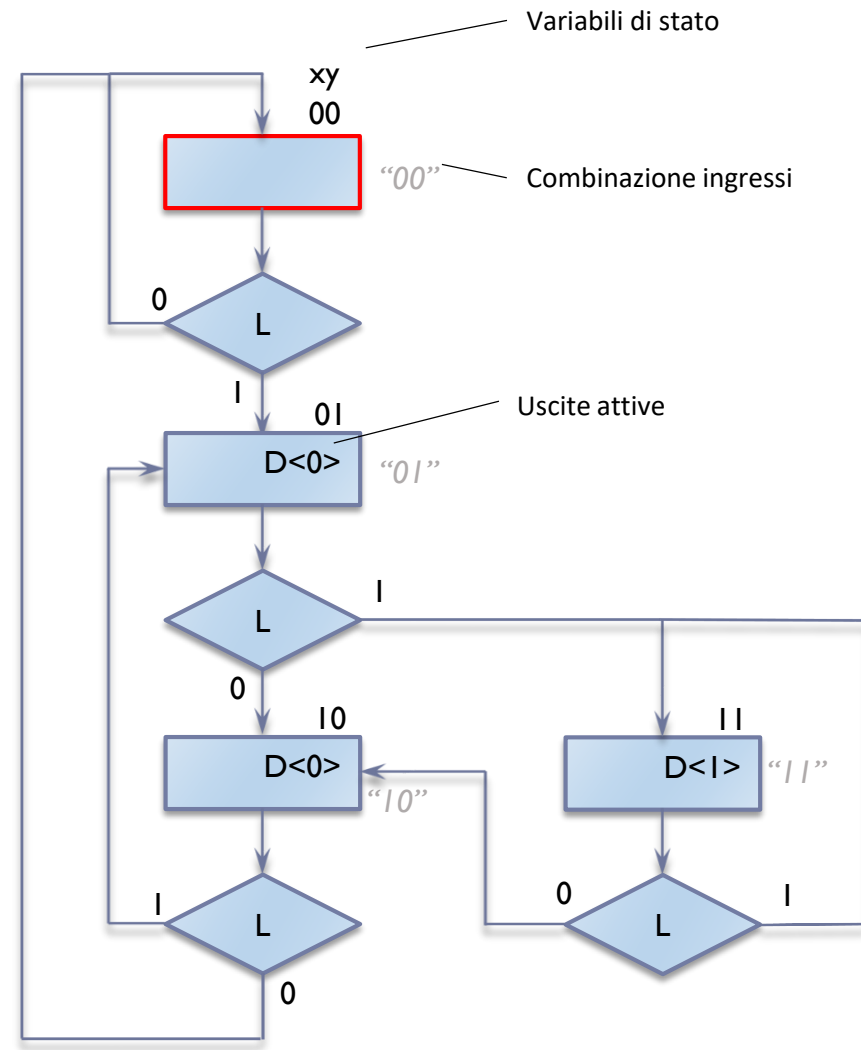
Contatore di uni

- Si progetti una macchina a stati finiti con
 - Un ingresso L seriale da un bit, sincrono con il clock
 - Due bit d'uscita $D<1:0>$
- Specifica
 - Le uscite devono, per ogni ciclo di clock, indicare in codice binario il numero di uni che si sono presentati all'ingresso L nei due cicli di clock precedenti
- VHDL
 - Si implementi la macchina a stati in codice VHDL verificando il funzionamento con il file di testbench fornito.
- Extra:
 - Diagramma temporale → si ipotizzi un andamento per L e si derivi il relativo diagramma temporale, indicando il valore dello **stato presente**, dello **stato futuro** e delle **uscite**
 - Circuito → si derivi il circuito, passando per le tabelle degli stati



Contatore di uni

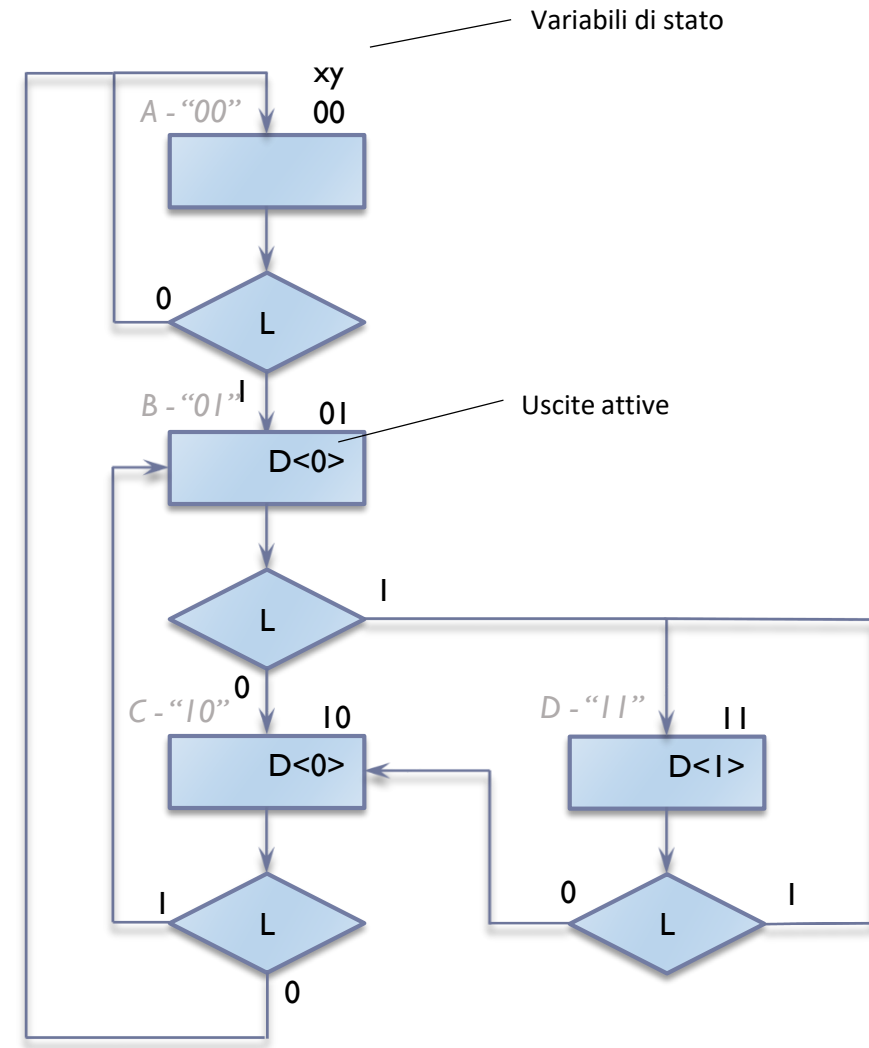
- Diagramma degli stati
 - Devo tenere traccia degli "1" negli ultimi 2 cicli di clock.
 - 00 → non ci sono "1"
 - 01 → solo un "1"
 - 10 → solo un "1"
 - 11 → due "1"
- Ho 4 possibili combinazioni
 - 4 stati codificabili in 2 variabili di stato x e y



Contatore di uni

- Tabella degli stati

<i>Ingresso</i>	<i>Stato presente</i>		<i>Stato futuro</i>		<i>Uscite</i>	
	x	y	x	y	D<1>	D<0>
L	x	y	x	y	D<1>	D<0>
0	0	0	0	0	0	0
0	0	1	1	0	0	1
0	1	0	0	0	0	1
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	0



Contatore di uni

- Sintesi delle uscite :
 - Dipendono solamente dallo stato presente e non dall'ingresso (Macchina di Moore). Si può ignorare L nelle mappe di Karnaugh.

Ingresso	Stato presente		Stato futuro		Uscite	
	x	y	x	y	D<1>	D<0>
L	x	y	x	y	D<1>	D<0>
0	0	0	0	0	0	0
0	0	1	1	0	0	1
0	1	0	0	0	0	1
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	0

x \ y	0	1
0	0	1
1	1	0

$$D<0> = x \oplus y$$

x \ y	0	1
0	0	0
1	0	1

$$D<1> = xy$$

Contatore di uni

- Sintesi delle variabili di stato con flip-flop D.
 - Considero le colonne dello stato futuro come il risultato dell'equazione caratteristica per flip flop D $\rightarrow Q(n+1) = D$

Ingresso	Stato presente		Stato futuro		Uscite	
	x	y	x	y	D<1>	D<0>
0	0	0	0	0	0	0
0	0	1	1	0	0	1
0	1	0	0	0	0	1
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	0

$L \setminus xy$	00	01	11	10
0	0	1	1	0
1	0	1	1	0

$$x_{+1} = y$$

$L \setminus xy$	00	01	11	10
0	0	0	0	0
1	1	1	1	1

$$y_{+1} = L$$

Contatore di uni

- Codice VHDL: ContatoreUni.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ContatoreUni is
    Port (
        clock : in std_logic;
        reset  : in std_logic;
        L      : in std_logic;
        Dout   : out std_logic_vector(1 downto 0)
    );
end ContatoreUni;

architecture Behavioral of ContatoreUni is

    type state_type is ( A, B, C, D );
    signal present_state, next_state : state_type;
begin
    process ( clock, reset ) begin
        if reset = '0' then
            -- reset attivo basso
            present_state <= A;
        elsif rising_edge( clock ) then
            present_state <= next_state;
        end if;
    end process;
end process;
```

Contatore di uni

- Codice VHDL: ContatoreUni.vhd

```
process ( present_state, L ) begin
  next_state <= present_state;
  case present_state is
    when A =>
      if L = '1' then
        next_state <= B;
      end if;
    when B =>
      if L = '0' then
        next_state <= C;
      else
        next_state <= D;
      end if;
    when C =>
      if L = '0' then
        next_state <= A;
      else
        next_state <= B;
      end if;
    when D =>
      if L = '0' then
        next_state <= C;
      end if;
  end case;
end process;
```

Contatore di uni

- Codice VHDL: ContatoreUni.vhd

```
process ( present_state ) begin
  Dout <= (others => '0');
  if (present_state = B) or (present_state = C) then
    Dout <= "01";
  end if;
  if (present_state = D) then
    Dout <= "10";
  end if;

end process;
end Behavioral;
```

Contatore di uni

- Codice VHDL: testbench.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity testbench is
-- Port ( );
end testbench;

architecture Behavioral of testbench is

    component ContatoreUni is
        Port (
            clock : in std_logic;
            reset : in std_logic;
            L : in std_logic;
            Dout : out std_logic_vector( 1 downto 0 )
        );
    end component;

    signal clock, reset : std_logic;
    signal L : std_logic;
    signal Dout : std_logic_vector( 1 downto 0 );

begin

    uut : ContatoreUni port map (
        clock => clock,
        reset => reset,
        L => L,
        Dout => Dout
    );
```

```
process begin
    clock <= '0'; wait for 5 ns;
    clock <= '1'; wait for 5 ns;
end process;

process begin
    reset <= '0'; wait for 100 ns;
    reset <= '1'; wait;
end process;

uut_test_proc : process begin
    L <= '0';
    wait for 125 ns;
    L <= '1';
    wait for 10 ns;
    L <= '0';
    wait for 10 ns;
    L <= '1';
    wait for 20 ns;
    L <= '0';
    wait for 30 ns;
    wait;
end process;

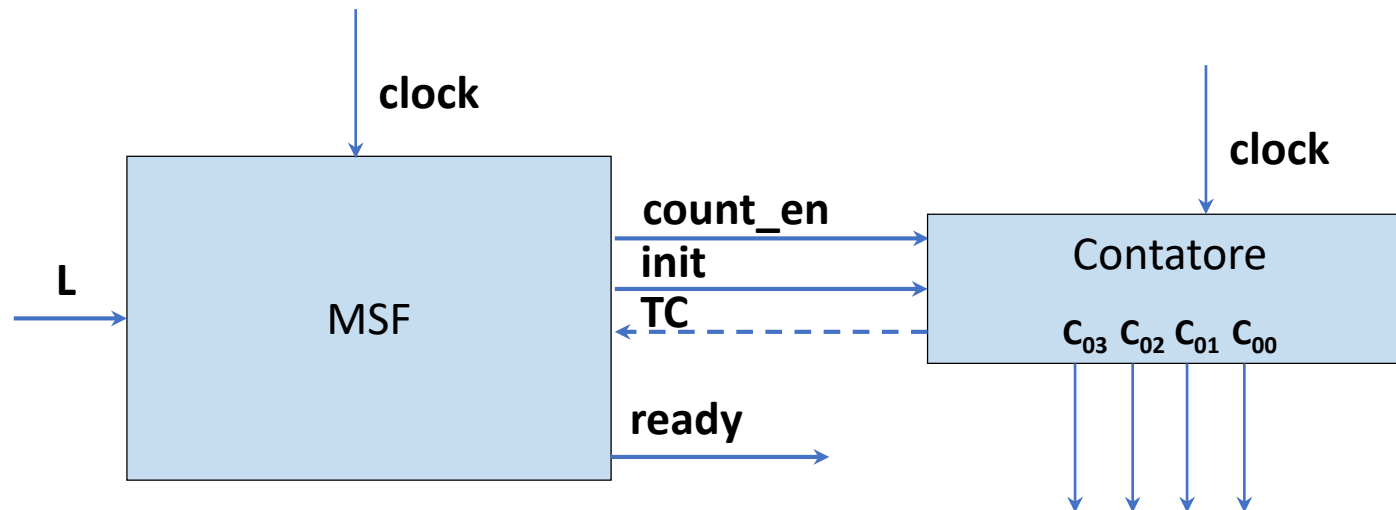
end Behavioral;
```

Misuratore di impulsi

- Si vuole realizzare un dispositivo che indichi la durata degli impulsi a 1 di un ingresso L
- A tal fine si utilizza un contatore dotato dei seguenti segnali
 - L'ingresso del clock
 - Un ingresso di abilitazione al conteggio (*count_en*), sincrono
 - Un ingresso di inizializzazione sincrono (*init*), che pone il contatore a 0
- Si realizzi una macchina a stati che
 - abiliti il contatore al conteggio quando il segnale L va a 1
 - blocchi il contatore quando il segnale L torna a 0, segnalando tale condizione attivando un segnale *ready*, che deve rimanere attivo per un ciclo di clock
 - una volta eseguito il conteggio, si predispone automaticamente per un nuovo conteggio
- Si trascurino eventuali condizioni di overflow del contatore
 - Alternativa: si rilevi il fondo scala tramite il segnale *terminal_count* (TC) del contatore e si blocchi il conteggio

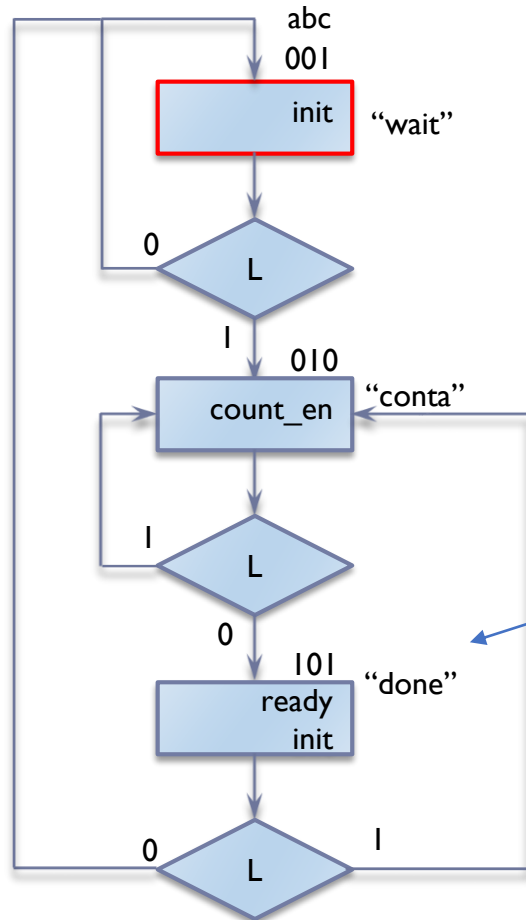
Misuratore di impulsi

- La MSF e il contatore sono pilotati dal medesimo clock
- Il tempo si conta in cicli di clock
- Il segnale *init* è sincrono, pertanto il contatore torna a zero nel ciclo di clock **successivo** a quello in cui *init* è attivo
- Anche *count_en* è sincrono, quindi il contatore avanza di 1 nel ciclo di clock dopo
- Fare il diagramma temporale (inventarsi un andamento di L) ed il circuito
- Si implementi in VHDL il circuito verificando il corretto funzionamento.



Misuratore di impulsi

- Diagramma degli stati



Stato wait

- Supponendo che l'ingresso L sia inizialmente a 0, attende che vada a 1
- Contemporaneamente inizializza il contatore

Stato conta (si entra con L = 1)

- Abilita il contatore al conteggio
- Attende che L torni a 0

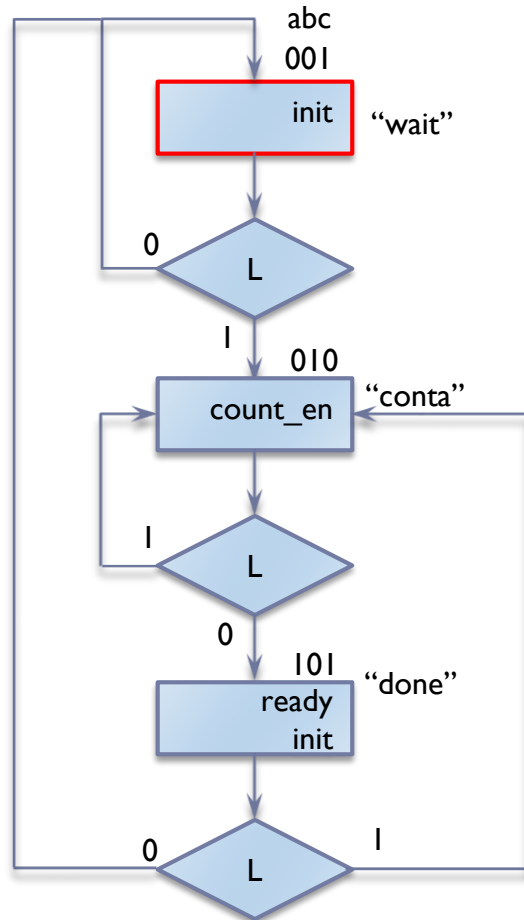
Stato done (si entra con L = 0)

- Fornisce il segnale di ready
- Ready attivo per un ciclo di clock
- L potrebbe andare subito a 1, quindi
 - Inizializza subito il contatore
 - Verifico il valore di L
 - Se L = 0, torno allo stato wait
 - Se L = 1, vado direttamente allo stato conta

- init è sincrono, quindi il contatore va a 0 solo nel ciclo di clock dopo
- In particolare, il contatore ha il valore corretto mentre si è in done e ready è attivo

Misuratore di impulsi

- Diagramma degli stati



Ci sono tre stati

- Possiamo usare semplicemente due variabili di stato
- Oppure seguire una codifica alternativa

Codifica one-hot

- Possibile, occorrono tre variabili di stato (abc)

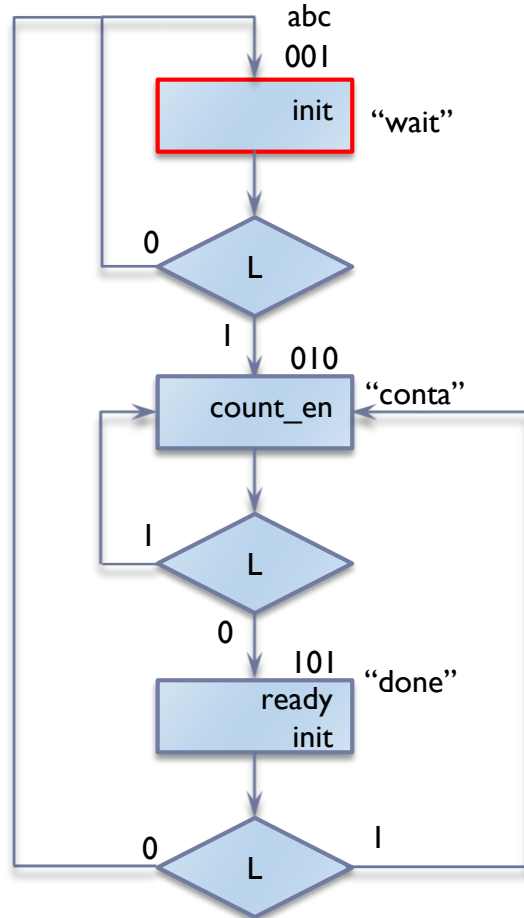
Variante sul one-hot ('semi one-hot')

- Abbiamo anche tre uscite
- Codifichiamo lo stato in modo che ogni variabile di stato corrisponda ad una uscita → non serve rete combinatoria per le uscite!
- Possibile perché non ci sono stati con la medesima combinazione delle uscite

ready = a
count_en = b
init = c

Misuratore di impulsi

• Sintesi



Mappa degli stati

$La \setminus bc$	00	01	11	10
00	-	wait	-	conta
01	-	done	-	-
11	-	done	-	-
10	-	wait	-	conta

Sintesi uscite

ready = a
count_en = b
init = c

Partendo dallo stato presente è possibile stabilire il valore che assumerà la variabile di stato osservando lo stato d'arrivo.

Sintesi variabili di stato

$La \setminus bc$	00	01	11	10
00	-	0	-	1
01	-	0	-	-
11	-	0	-	-
10	-	0	-	0

$$a_{+1} = L'b$$

$La \setminus bc$	00	01	11	10
00	-	0	-	0
01	-	0	-	-
11	-	1	-	-
10	-	1	-	1

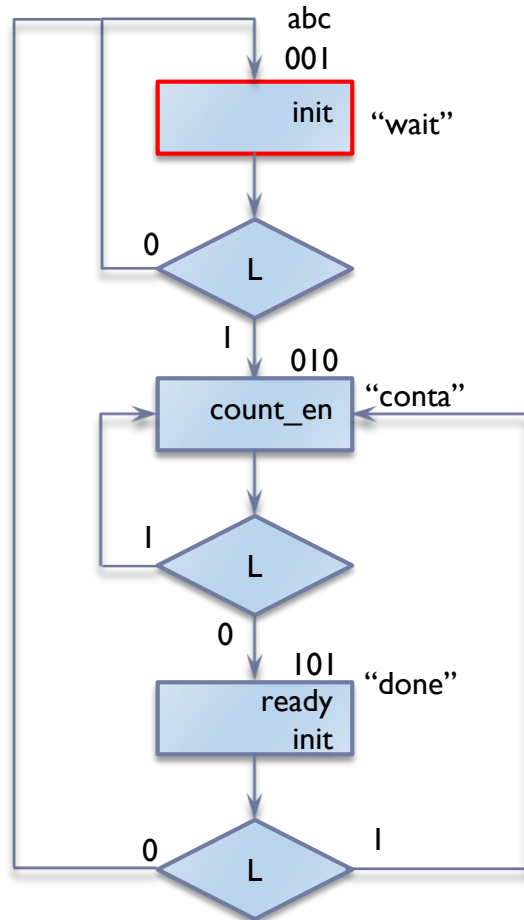
$$b_{+1} = L$$

$La \setminus bc$	00	01	11	10
00	-	1	-	1
01	-	1	-	-
11	-	0	-	-
10	-	0	-	0

$$c_{+1} = L'$$

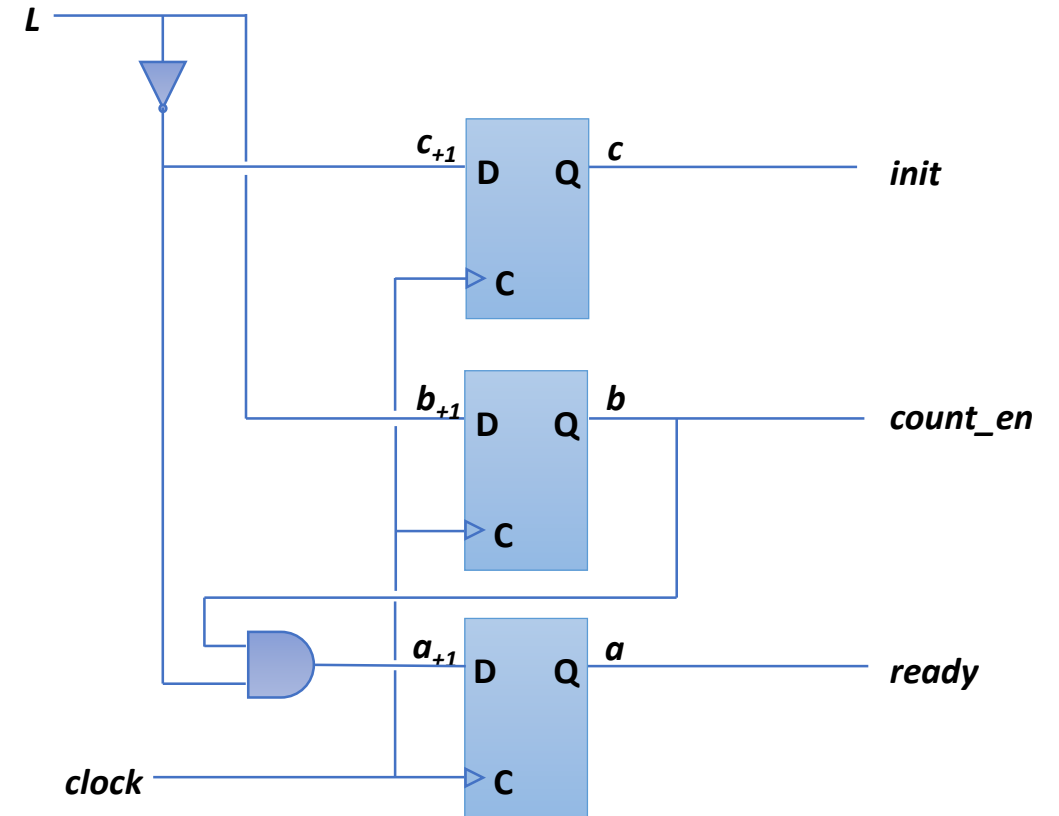
Misuratore di impulsi

- Schema circuitale

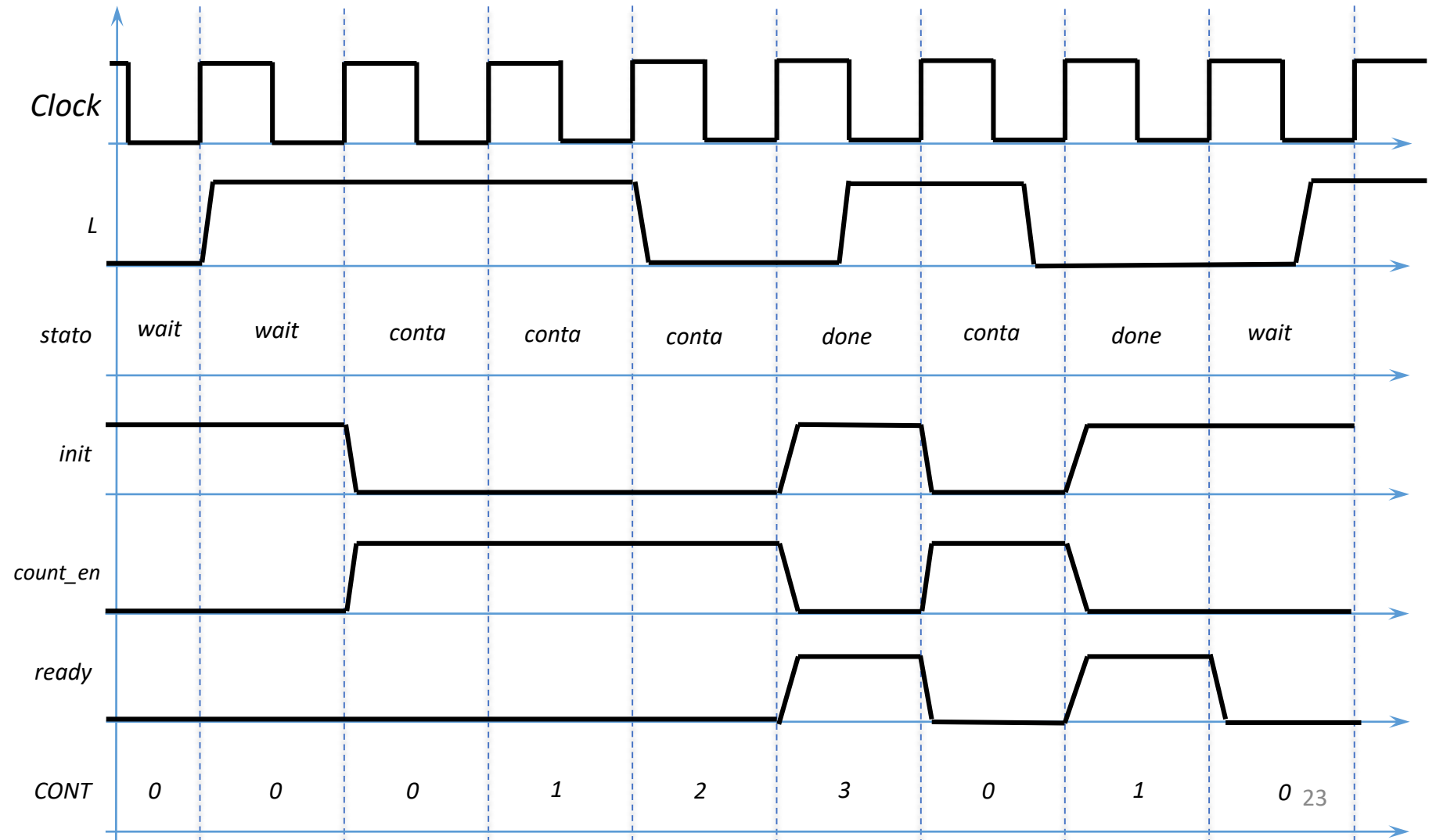
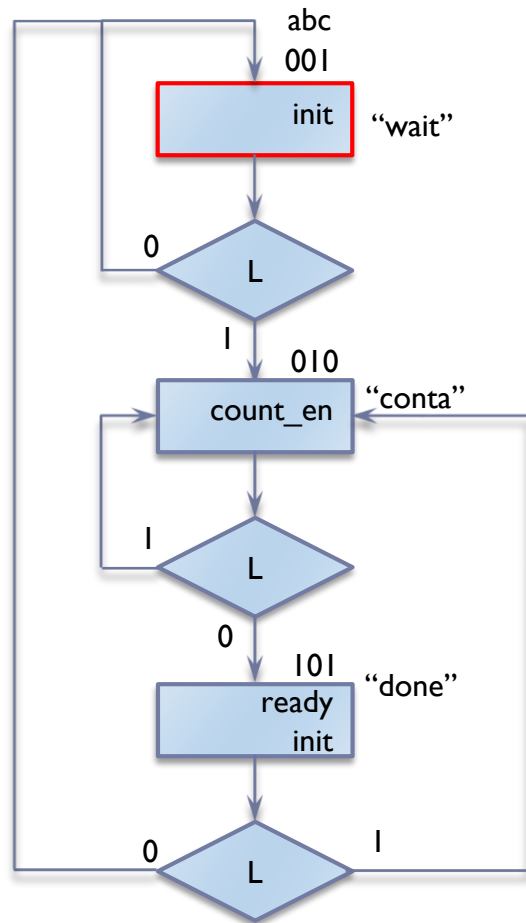


$a_{+1} = L'b$
 $b_{+1} = L$
 $c_{+1} = L'$

$init = c$
 $count_en = b$
 $ready = a$



- Diagramma temporale



Misuratore di impulsi

- Codice VHDL: MSF.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MSF is
    Port (
        clock : in std_logic;
        L : in std_logic;
        reset: in std_logic;
        count_en, init, ready : out std_logic
    );
end MSF;

architecture Behavioral of MSF is
    type state_type is ( at_wait, at_conta, at_done );
    signal present_state, next_state : state_type;

    Begin
        process ( clock, reset ) begin
            if reset = '1' then
                present_state <= at_wait;
            elsif rising_edge( clock ) then
                present_state <= next_state;
            end if;
        end process;
```

```
        process ( present_state, L ) begin
            next_state <= present_state;
            case present_state is
                when at_wait =>

                    if L = '1' then
                        next_state <= at_conta;
                    end if;
                when at_conta =>
                    if L = '0' then
                        next_state <= at_done;
                    end if;
                when at_done =>
                    if L = '0' then
                        next_state <= at_wait;
                    else
                        next_state <= at_conta;
                    end if;
            end case;
        end process;
```

Misuratore di impulsi

- Codice VHDL: MSF.vhd

```
process ( present_state ) begin

    init <= '0';
    count_en <= '0';
    ready <= '0';

    if present_state = at_wait then
        init <= '1';
    end if;
    if present_state = at_conta then
        count_en <= '1';
    end if;
    if present_state = at_done then
        ready <= '1';
        init <= '1';
    end if;
end process;

end Behavioral;
```


Misuratore di impulsi

- Codice VHDL: Contatore.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Contatore is
port (
    res, clock, count_en, init: in std_logic;
    y : buffer unsigned(3 downto 0)
);
end entity Contatore;

architecture behavioral of Contatore is
begin
    process (res, clock) is
    begin
        if (res = '1') then
            y <= (others => '0');
        elsif rising_edge(clock) then
            if init = '1' then
                y <= (others => '0');
            elsif count_en = '1' then
                y <= y + 1;
            end if;
        end process;
    end architecture behavioral;
```

Misuratore di impulsi

- Codice VHDL: MisuratoreImpulsi.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity MisuratoreImpulsi is
    Port (
        clock, L: in std_logic;
        c: buffer std_logic_vector(3 downto 0);
        ready: out std_logic
    );
end MisuratoreImpulsi;

architecture Behavioral of MisuratoreImpulsi is
    signal init, count_en : std_logic;
begin
```

```
MacchinaStati : entity work.MSF
    port map (
        clock => clock,
        L => L,
        init => init,
        ready => ready,
        reset => reset,
        count_en => count_en
    );

Contatore : entity work.Contatore
    port map (
        res => reset,
        init => init,
        clock => clock,
        count_en => count_en,
        y => c
    );

end Behavioral;
```

Misuratore di impulsi

- Codice VHDL: MisuratoreImpulsi_testbench.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity MisuratoreImpulsi_testbench is
-- Port ( );
end MisuratoreImpulsi_testbench;

architecture Behavioral of MisuratoreImpulsi_testbench is

    component MisuratoreImpulsi is
        Port (
            clock , L, reset: in std_logic;
            c: buffer unsigned(3 downto 0);
            ready: out std_logic
        );
    end component;

    signal clock : std_logic;
    signal reset : std_logic;
    signal L : std_logic;
    signal c : unsigned(3 downto 0);
    signal ready : std_logic;

begin
```

```
    uut : MisuratoreImpulsi
        port map(
            clock => clock,
            L => L,
            c => c,
            ready => ready,
            reset => reset
        );

    process begin
        clock <= '0'; wait for 5 ns;
        clock <= '1'; wait for 5 ns;
    end process;

    process begin
        reset <= '1'; wait for 100 ns;
        reset <= '0'; wait;
    end process;

    uut_test_proc : process
    begin

        L <= '0';          wait for 125 ns;
        L <= '1';          wait for 30 ns;
        L <= '0';          wait for 15 ns;
        L <= '1';          wait for 50 ns;
        L <= '0';          wait for 30 ns;
        L <= '1';          wait for 100 ns;
        L <= '0';          wait for 20 ns;

    wait;
    end process;

end Behavioral;
```